Linux Shell Programming

Hands-On Guide | Ubuntu on WSL

Practical examples: your first Bash script, Linux permissions, shell variables and keyboard input.

Example 1 — Create and run hello.sh

1. Open Ubuntu. Work inside the Linux home directory rather than /mnt/c when practising chmod.

```
cd ~
mkdir -p ShellLab
cd ShellLab
pwd
```

Expected path: **/home/ubuntu/ShellLab**

2. Create the script with **nano hello.sh** and enter:

```
#!/bin/bash

echo "Welcome to Linux Shell Programming"
echo "My first shell script"
date
```

Save: Ctrl+O, Enter; exit: Ctrl+X.

3. Inspect, grant execute permission and run:

```
ls -l hello.sh
chmod 755 hello.sh
ls -l hello.sh
cat hello.sh
./hello.sh
```

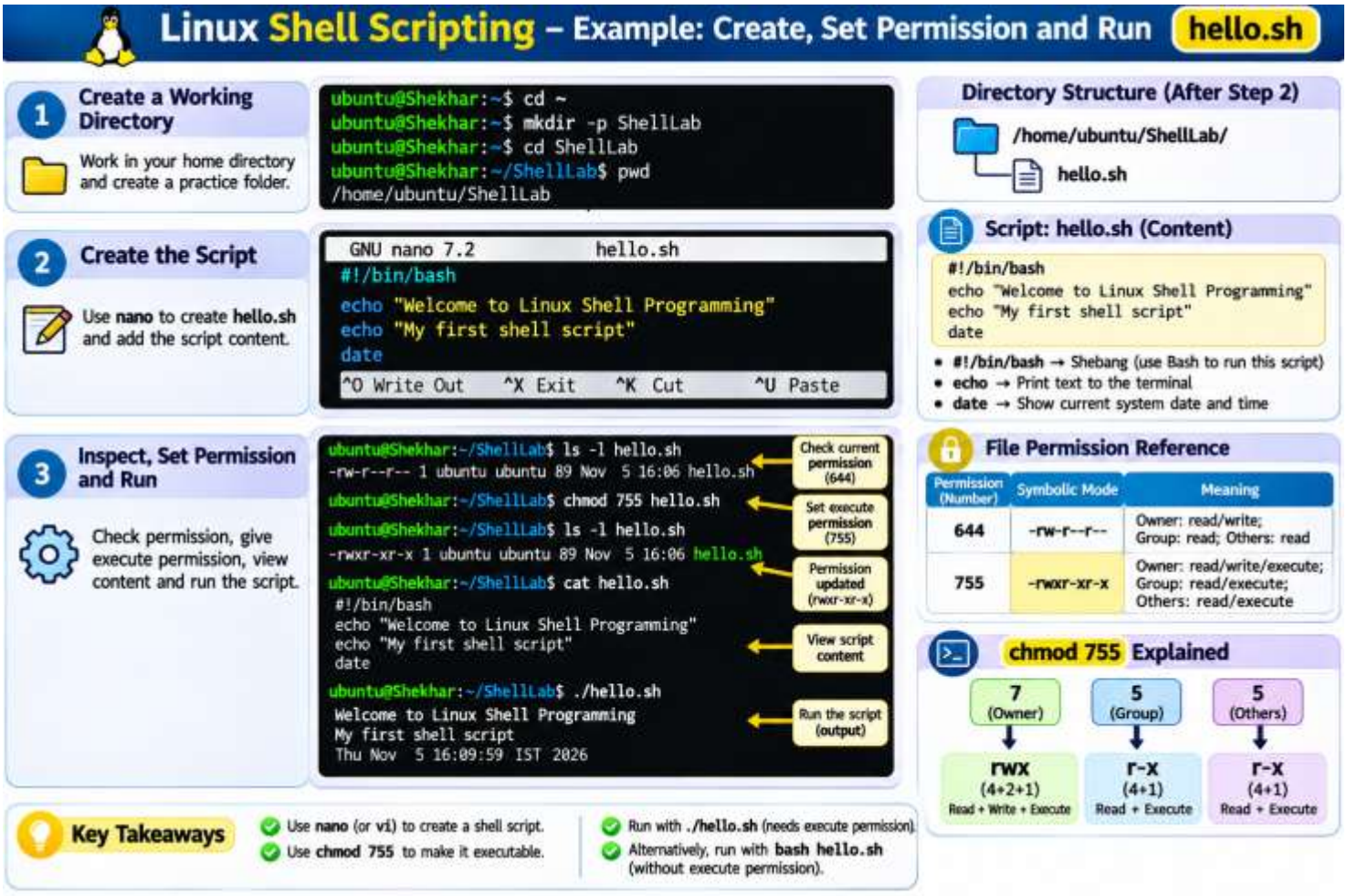
Observed output from your Ubuntu session:

-rwxr-xr-x 1 ubuntu ubuntu 89 Nov 5 16:06 hello.sh
Welcome to Linux Shell Programming
My first shell script
Thu Nov 5 16:09:59 IST 2026

Note: date output changes each time the script runs. You can also use bash hello.sh without execute permission.

Understand file permissions

Permission	Meaning
644 / -rw-r--r--	Owner: read/write; group: read; others: read
755 / -rwxr-xr-x	Owner: read/write/execute; group: read/execute; others: read/execute
chmod 755 hello.sh	Set mode 755 on hello.sh
./hello.sh	Run executable script from current folder



Example 2 — Variables and user input

1. Create a second script:

cd ~/ShellLab

nano student.sh

2. Enter this Bash code:

```
#!/bin/bash

echo "Enter student name:"
read name

echo "Enter student age:"
read age

echo "Enter course name:"
read course

echo "------ Student Details ------"
echo "Name: $name"
echo "Age: $age"
echo "Course: $course"
```

3. Save and run:

```
chmod 755 student.sh
./student.sh
```

Sample interactive session:

```
Enter student name:
Rahul
Enter student age:
25
```

Enter course name:

Linux

------ Student Details ------

Name: Rahul

Age: 25

Course: Linux

Key commands explained

#!/bin/bash	Run the script using Bash
echo "text"	Print text to the terminal
read name	Read keyboard input into the name variable
$name	Substitute the stored value of name
nano student.sh	Open/create script in the nano editor
bash student.sh	Run with Bash; execute bit not required

Practice tasks

1. Change the greeting in hello.sh and rerun it.
2. Add a city variable to student.sh and display it.
3. Try bash student.sh after removing execute permission with chmod 644 student.sh.
4. Use ls -l to compare modes 644 and 755.

Linux Shell Scripting – Example 3: Decision, Loop and Function

A. Decision Making (if-else)

Objective

Take a number as input and check if it is even or odd.

File: even_odd.sh

Script: even_odd.sh

```
#!/bin/bash
echo "Enter a number:"
read num
if (( num % 2 == 0 )); then
    echo "$num is Even"
else
    echo "$num is Odd"
fi
```

Run & Output

```
ubuntu@Shekhar:~/ShellLab$ ./even_odd.sh
Enter a number:
10
10 is Even
ubuntu@Shekhar:~/ShellLab$ ./even_odd.sh
Enter a number:
7
7 is Odd
```

Key Takeaways

- **if** – checks a condition.
- **then** – runs if the condition is true.
- **else** – runs if the condition is false.
- **fi** – ends the if-else block.
- **((num % 2 == 0))** – arithmetic check for even number.

B. Loop (for)

Objective

Print numbers from 1 to 5 using a for loop.

File: loop.sh

Script: loop.sh

```
#!/bin/bash
echo "Numbers from 1 to 5:"
for i in {1..5}; do
    echo "Number: $i"
done
```

Run & Output

```
ubuntu@Shekhar:~/ShellLab$ ./loop.sh
Numbers from 1 to 5:
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

Key Takeaways

- **for** – repeats a block of commands.
- **{1..5}** – generates numbers 1 to 5.
- **do** – starts the loop body.
- **done** – ends the loop.
- **$i** – loop variable (current number).

C. Function

Objective

Use a function to greet the user by name.

File: greet.sh

Script: greet.sh

```
#!/bin/bash
greet() {
  echo "Hello, $1! Welcome to Linux."
}
echo "Enter your name:"
read name
greet "$name"
```

Run & Output

```
ubuntu@Shekhar:~/ShellLab$ ./greet.sh
Enter your name:
Rahul
Hello, Rahul! Welcome to Linux.
```

Key Takeaways

- A function is a reusable block of code.
- **greet() { ... }** – defines a function.
- **$1** – the first argument passed to the function.
- **greet "$name"** – calls the function with user input.

Other Useful Loop Variations

Loop Type	Example
for (list)	`for item in a b c; do ... done`
while	`while [ condition ]; do ... done`
until	`until [ condition ]; do ... done`

Common Operators in if Condition

Operator	Meaning
-eq	Equal to (numbers)
-ne	Not equal to
-gt	Greater than
-lt	Less than
-ge	Greater than or equal to
-le	Less than or equal to

Hands-on Practice

1. Modify **even_odd.sh** to check if a number is positive, negative or zero.
2. Update **loop.sh** to print the square of numbers from 1 to 5.
3. Create a function in **greet.sh** to show a welcome message with date.

Linux Shell Scripting – Example 3: if-else, for Loop and Function

A. Decision Making (if-else)

Objective: Check a student's marks and print Pass or Fail.

Script: result.sh

```
#!/bin/bash
echo "Enter student marks:"
read marks
if [ "$marks" -ge 40 ]; then
    echo "Result: PASS"
else
    echo "Result: FAIL"
fi
```

Run & Output

```
ubuntu@Shekhar:~/ShellLab$ ./result.sh
Enter student marks:
75
Result: PASS
```

if-else Flow



B. Loop (for)

Objective: Print numbers from 1 to 5 using a for loop.

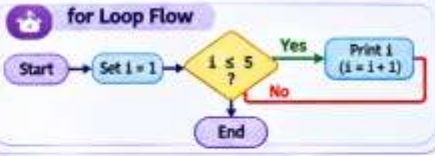
Script: loop.sh

```
#!/bin/bash
echo "Numbers from 1 to 5:"
for i in {1..5}; do
    echo "Number: $i"
done
```

Run & Output

```
ubuntu@Shekhar:~/ShellLab$ ./loop.sh
Numbers from 1 to 5:
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

for Loop Flow



C. Function

Objective: Use a function to greet the user by name.

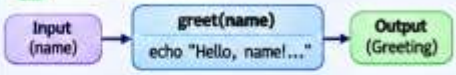
Script: greet.sh

```
#!/bin/bash
greet() {
    echo "Hello, $1! Welcome to Linux."
}
echo "Enter your name:"
read name
greet "$name"
```

Run & Output

```
ubuntu@Shekhar:~/ShellLab$ ./greet.sh
Enter your name:
Rahul
Hello, Rahul! Welcome to Linux.
```

Function Flow



Steps to Create & Run a Script

1	`cd ~/ShellLab`	→ Go to working directory
2	`nano script.sh`	→ Create / open the script
3	Write the code	→ Add if-else / for / function
4	`chmod 755 script.sh`	→ Give execute permission
5	`./script.sh`	→ Run the script

Key Commands & Operators

Command / Operator	Meaning
`#!/bin/bash`	Run the script using Bash
`echo "text"`	Print text to the terminal
`read var`	Read user input into a variable
`[ "$a" -ge 40 ]`	Check if a is greater than or equal to 40
`for i in {1..5}`	Loop from 1 to 5
`function() { ... }`	Define a reusable block of code

Operator (for Conditions) Quick Reference

Operator	Meaning	Example
`-eq`	Equal to	`[ $a -eq 10 ]`
`-ne`	Not equal to	`[ $a -ne 10 ]`
`-gt`	Greater than	`[ $a -gt 10 ]`
`-ge`	Greater than or equal to	`[ $a -ge 10 ]`
`-lt`	Less than	`[ $a -lt 10 ]`
`-le`	Less than or equal to	`[ $a -le 10 ]`

Key Takeaways

- Use **if-else** for decision making.
- Use **for loops** to repeat commands.
- Use **functions** to write reusable code.
- Give execute permission with **chmod 755**.
- Run scripts using **./script.sh** or **bash script.sh**.

Linux Shell Scripting – Example 4: Arrays, Functions & File Operations

A. Arrays in Bash

Objective: Store multiple values and access them using an array.

Script: array.sh

```
#!/bin/bash
# Define an array
fruits=("Apple" "Banana" "Mango" "Orange")
echo "All Fruits: ${fruits[@]}"
echo "First Fruit: ${fruits[0]}"
echo "Total Fruits: ${#fruits[@]}"
```

Run & Output

```
ubuntu@Shekhar:~/ShellLab$ ./array.sh
All Fruits: Apple Banana Mango Orange
First Fruit: Apple
Total Fruits: 4
```

B. Function with Parameters

Objective: Create a function that adds two numbers.

Script: add.sh

```
#!/bin/bash
add() {
    sum=$(( $1 + $2 ))
    echo "Sum: $sum"
}
# Call the function with two arguments
add 10 20
```

Run & Output

```
ubuntu@Shekhar:~/ShellLab$ ./add.sh
Sum: 30
```

C. File Operations (Check & Count)

Objective: Check if a file exists and count the number of lines.

Script: file_ops.sh

```
#!/bin/bash
file="data.txt"
if [ -f "$file" ]; then
    echo "File exists: $file"
    lines=$(wc -l < "$file")
    echo "Total Lines: $lines"
else
    echo "File not found: $file"
fi
```

Run & Output

```
ubuntu@Shekhar:~/ShellLab$ ./file_ops.sh
File exists: data.txt
Total Lines: 3
```

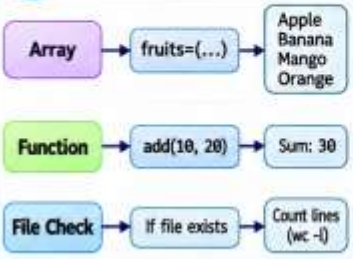
& Common File Operations in Bash

Command	Purpose
`touch file.txt`	Create an empty file
`cat file.txt`	Display file contents
`wc -l file.txt`	Count lines in a file
`cp file.txt backup.txt`	Copy a file
`mv file.txt new.txt`	Rename or move a file
`rm file.txt`	Delete a file
`rm -r folder/`	Delete a directory (recursively)
`ls -l`	List files with details

Key Concepts

- **Array:** Store multiple values (e.g., fruits).
- `${array[@]}`: Get all elements of the array.
- `${array[0]}` : Get the first element.
- **Function:** Reusable block of code (e.g., add()).
- `$1, $2` : Positional parameters to a function.
- `-f` : Check if a file exists and is a regular file.
- `wc -l` : Count the number of lines in a file.

Array, Function & File Flow



Hands-on Practice

1. Create data.txt with 3–4 lines and run **file_ops.sh**.
2. Modify **add.sh** to perform subtraction and multiplication.
3. Create a new script **menu.sh** that shows a menu:
   1. Show current date
   2. List files in current directory
   3. Exit

Linux Shell Scripting – Example 5: While Loop, Arrays, Functions & File Operations

A. While Loop – Print 1 to 5

Objective:
Print numbers from 1 to 5 using a while loop.

1 Create the Script

```
cd ~/ShellLab
nano while.sh
```

Bash Code

```
#!/bin/bash
count=1
while [ "$count" -le 5 ]
do
    echo "Number: $count"
    count=$((count + 1))
done
```

3 Run the Script

```
chmod 755 while.sh
./while.sh
```

Output:

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

How It Works?

- Start with count = 1
- Continue while count ≤ 5
- Increase count by 1
- Stop when count becomes 6

B. Arrays – Store Multiple Values

Objective: Store fruits in an array and display them.

1 Script: array.sh

```
#!/bin/bash
fruits=("Apple" "Banana" "Mango")
echo "All Fruits: ${fruits[@]}"
echo "First Fruit: ${fruits[0]}"
echo "Total Fruits: ${#fruits[@]}"
```

Output:

```
All Fruits: Apple Banana Mango
First Fruit: Apple
Total Fruits: 3
```

C. Functions – Reusable Code

Objective: Create a function to greet the user.

1 Script: greet.sh

```
#!/bin/bash
greet() {
    echo "Hello, $1! Welcome to Linux."
}
greet "Rahul"
```

Output:

```
Hello, Rahul! Welcome to Linux.
```

D. File Operations – Check & Count Lines

Objective: Check if a file exists and count the number of lines.

1 Script: file_ops.sh

```
#!/bin/bash
file="data.txt"
if [ -f "$file" ]; then
    echo "File exists: $file"
    lines=$(wc -l < "$file")
    echo "Total Lines: $lines"
else
    echo "File not found: $file"
fi
```

Create a File

```
echo "Line1" > data.txt
echo "Line2" >> data.txt
echo "Line3" >> data.txt
```

Output:

```
File exists: data.txt
Total Lines: 3
```

Key Takeaways

- **While Loop:** Repeat actions while a condition is true.
- **Arrays:** Store multiple values in a single variable.
- **Functions:** Write reusable code with parameters.
- **File Operations:** Check files and process data (e.g., count lines).

Practice these concepts and build more useful scripts!

Linux Shell Scripting – Example 2: Variables and User Input `student.sh`

Objective

Create a shell script that takes user input (name, age, course) using variables and displays the details.

Steps Overview

1. Go to **~/ShellLab**
2. Create script with **nano**
3. Write the code
4. Save & exit (Ctrl+O, Enter, Ctrl+X)
5. Give execute permission (**chmod 755**)
6. Run the script (**./student.sh**)

Step-by-Step Commands and Output

1. Go to the working directory

```
ubuntu@Shekhar:~$ cd ~/ShellLab
ubuntu@Shekhar:~/ShellLab$
```

2. Create / open student.sh in nano

```
ubuntu@Shekhar:~/ShellLab$ nano student.sh
```

3. Give execute permission (755)

```
ubuntu@Shekhar:~/ShellLab$ chmod 755 student.sh
```

4. Interactive session (output)

```
ubuntu@Shekhar:~/ShellLab$ ./student.sh
Enter student name:
Rahul
Enter student age:
25
Enter course name:
Linux
------ Student Details ------
Name: Rahul
Age: 25
Course: Linux
```

Script: student.sh (Code)

```
#!/bin/bash

echo "Enter student name:"
read name

echo "Enter student age:"
read age

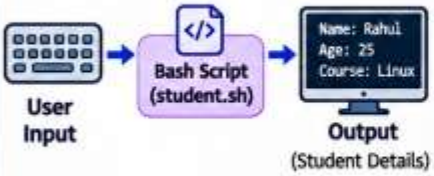
echo "Enter course name:"
read course

echo "------ Student Details ------"
echo "Name: $name"
echo "Age: $age"
echo "Course: $course"
```

Permissions: 755

User	Value	Permission	Meaning
Owner	7	rwx	Read + Write + Execute
Group	5	r-x	Read + Execute
Others	5	r-x	Read + Execute

Understanding the Script



Key Commands and Their Meaning

Command / Element	Meaning
`#!/bin/bash`	Run the script using Bash.
`echo "text"`	Print text to the terminal.
`read name`	Read keyboard input into the variable name.
`$name`	Substitute the stored value of name.
`nano student.sh`	Open/create the script in the nano editor.
`chmod 755 student.sh`	Set execute permission (rwxr-xr-x).
`./student.sh`	Run the script from the current folder (execute bit required).
`bash student.sh`	Run with Bash (execute bit not required).

Key Takeaways

- Use **read** to take input from the user.
- Store input in variables (name, age, course).
- Use **$variable** to print the stored values.
- Make the script executable with **chmod 755**.
- Run using **./student.sh** or **bash student.sh**.